

Link do github:

<https://github.com/Kamiks0320/Detekcja-Gestow>

Wizja komputerowa projekt - Raport techniczny

Temat: Rozpoznawanie statycznych gestów dłoni

Prowadzący przedmiot: dr inż. Joanna Marnik

Skład zespołu:

Marcin Wiśniowski 177177

Sławomir Zdunek 177192

Kamil Węgrzyn 177174

Kamil Śliwa 177165

3EF-DI/AA L02

Spis treści:

Spis treści:	1
1. Wstęp	2
1.1. Cel projektu	2
1.2. Opis problemu	2
1.3 Architektura systemu	3
2. Zastosowane metody	4
a) Konwersja BGR na HSV	4
b) Modelowanie koloru skóry w HSV	4
c) Detekcja konturów - Contour Detection	5
d) Wybór największego konturu	5
e) Wyznaczenie otoczki wypukłej - Convex Hull	5
f) Wyznaczanie defektów wypukłości - Convexity Defects	5
g) Operacja morfologiczna otwarcia	6
h) Operacja morfologiczna zamknięcia	6
i) Ekstrakcja deskryptorów geometrycznych	6
j) Normalizacja	7
k) Obliczenie odległości euklidesowej	7
l) Klasyfikacja gestu metodą k-NN	8
m) Walidacja modelu	8
3. Opis danych	9
4. Eksperymenty i analiza wyników	10
4.1 Oryginalne obrazy	10
4.2 Oryginalne maski	13
4.3 Przycięte obrazy	14
4.4 Przycięte maski	15
4.5 Przycięte obrazy + usunięcie problematycznych zdjęć	17
4.6 Trening na bazie, test na danych z zewnątrz - bez usuwania problematycznych zdjęć	18
4.7 Trening na bazie, test na danych z zewnątrz - jako maski	20
5. Wnioski	22
6. Bibliografia	23

1. Wstęp

1.1. Cel projektu

Celem projektu jest opracowanie systemu rozpoznawania statycznych gestów dłoni dla wybranych gestów: róg, okej, L, palec wskazujący w górę i kciuk w górę. System ma umożliwiać klasyfikację gestów na podstawie obrazów pochodzących z bazy danych

W ramach projektu został zrealizowany kompletny proces przetwarzania obrazu obejmujący:

- wstępne przetwarzanie obrazu,
- segmentację dłoni,
- wyznaczenie obszaru zainteresowania (ROI),
- ekstrakcję cech,
- klasyfikację gestów,
- wizualizację wyników.

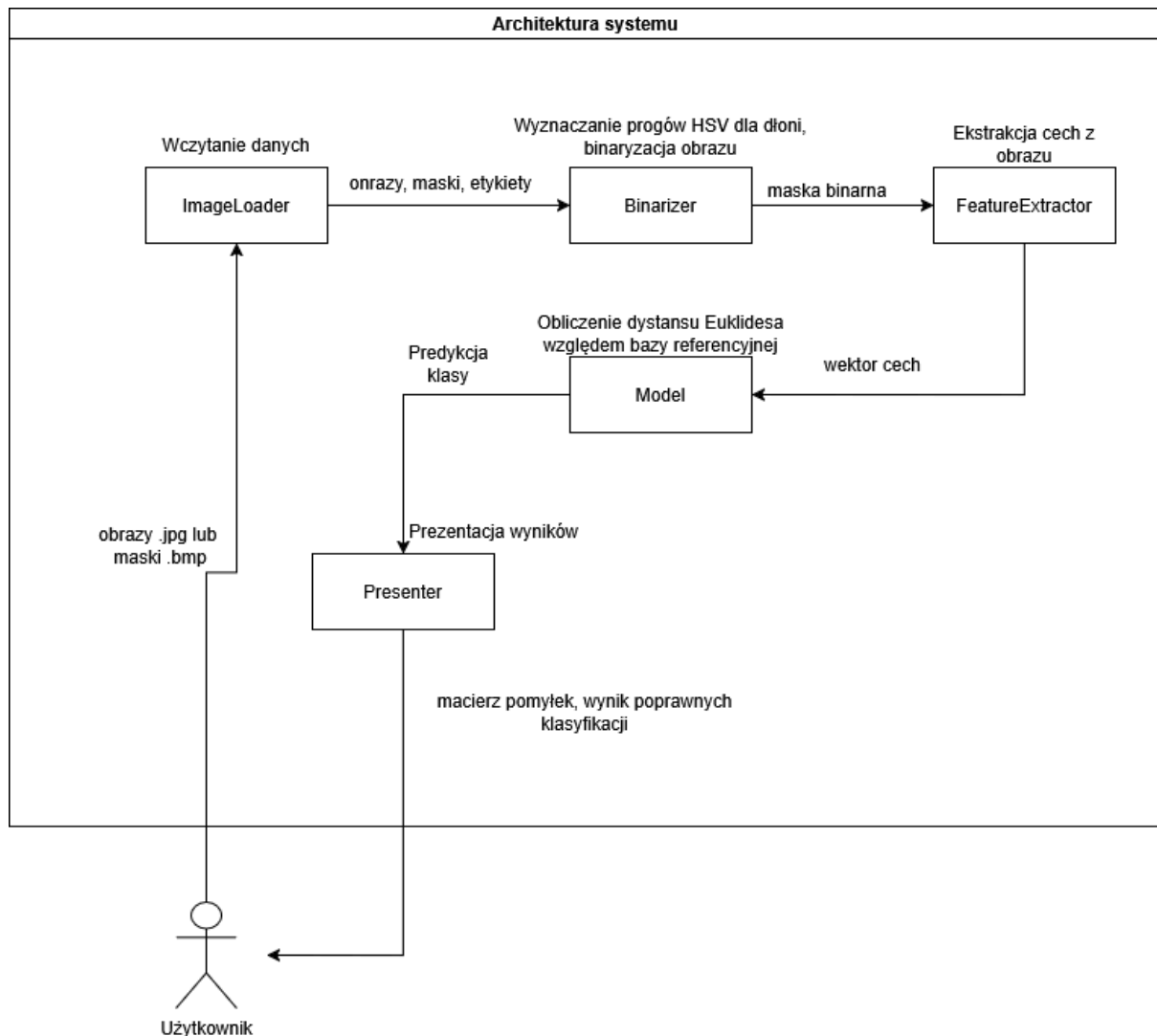
Głównym celem badawczym jest porównanie skuteczności klasycznych metod wizji komputerowej z podejściem opartym na sieci głębokiej YOLOv8. Dodatkowo przeprowadzone zostaną eksperymenty pozwalające ocenić wpływ kadrowania obrazów, jakości danych oraz usunięcia problematycznych przykładów na końcową skuteczność klasyfikacji.

1.2. Opis problemu

Rozpoznawanie statycznych gestów dłoni jest zadaniem polegającym na przypisaniu obrazu dłoni do odpowiedniej klasy gestu na podstawie cech wyekstraktowanych z obrazu. Realizacja takiego zadania wymaga poprawnego wykrycia dłoni na obrazie, wyodrębnienia jej z tła oraz znalezienia parametrów pozwalających odróżnić poszczególne gesty.

Problem jest utrudniony przez zmienne warunki oświetleniowe, różnorodne tło oraz różnice w sposobie wykonywania gestów przez różne osoby. Dodatkową trudność stanowi obecność obiektów o kolorze zbliżonym do skóry, które mogą powodować błędy podczas segmentacji dłoni.

1.3 Architektura systemu



Powyższy rysunek przedstawia uproszczony model systemu. Po uruchomieniu programu najpierw tworzony jest obiekt **ImageLoader**, który odpowiada za pobranie danych z folderów z obrazami i maskami. Następnie wywoływane jest **load_all()**, które wczytuje wszystkie obrazy **.jpg**, dobiera do nich odpowiadające maski **.bmp** i przypisuje etykiety klas na podstawie nazw plików.

Następnie tworzymy obiekt **Model**. Do modelu przekazywane są obrazy, maski i etykiety. W konstruktorze modelu najpierw wyznaczany jest wspólny zakres koloru skóry w przestrzeni HSV (metody z obiektu **Binarizer**). Program bierze piksele dłoni z masek referencyjnych i na tej podstawie ustala dolny oraz górny zakres HSV, który później służy do segmentacji dłoni na nowych obrazach. Potem dane są dzielone na część treningową i testową.

Dla każdej maski treningowej wywoływana jest funkcja `extract_features()`. Program znajduje największy kontur, oblicza otoczkę wypukłą, defekty wypukłości i deskryptory geometryczne. Z tego powstaje wektor cech opisujący kształt dłoni. Ten wektor razem z etykietą gestu trafia do bazy cech modelu.

Gdy chcemy sklasyfikować nowy obraz, wywoływana jest metoda `Classify()`. Najpierw obraz jest konwertowany do HSV, potem na podstawie wcześniej ustalonego zakresu HSV tworzona jest maska dłoni. Na tej masce wykonywane są operacje morfologiczne, żeby ograniczyć szum i poprawić kształt obiektu.

Następnie z utworzonej maski ponownie wyciągane są cechy geometryczne. Otrzymany wektor cech jest porównywany ze wszystkimi wektorami zapisanymi wcześniej w bazie treningowej. Do porównania używana jest suma kwadratów różnic, czyli uproszczona odległość euklidesowa.

Po obliczeniu odległości program sortuje próbki od najbardziej podobnych do najmniej podobnych. Następnie bierze cztery najbliższe próbki, czyli działa jak klasyfikator k-NN z $k = 4$. Wynikiem klasyfikacji jest ta klasa, która najczęściej występuje wśród tych czterech najbliższych sąsiadów.

2. Zastosowane metody

Do użytych metod należą:

a) Konwersja BGR na HSV

Zmiana przestrzeni barw na HSV, która jest bardziej odporna na zmiany oświetlenia przy segmentacji skóry niż wartości BGR.

W kodzie konwersja wykonywana jest za pomocą funkcji:

```
image_hsv = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2HSV)
```

Dzięki temu dalsze progowanie obrazu odbywa się na podstawie składowych H, S i V.

b) Modelowanie koloru skóry w HSV

Przed pobraniem pikseli dokonywana jest erozja maskii elementem 5×5 , aby pobierać piksele głównie ze środka dłoni, a nie z niedokładnych krawędzi maski.

Automatyczne wyznaczenie zakresu HSV skóry na podstawie masek referencyjnych i percentyli (5% i 95%) kolorów dłoni.

Percentyle 5% i 95% zastosowano, aby pominąć skrajne wartości HSV wynikające z cieni, odbić światła, szumu lub niedokładnych krawędzi maski.

c) Detekcja konturów - Contour Detection

Znalezienie i wyznaczenie granic obiektów na obrazie binarny.

W projekcie wykorzystywana jest funkcja:

```
cv2.findContours(mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

Parametr RETR_EXTERNAL oznacza, że wyszukiwane są tylko kontury zewnętrzne. Jest to wystarczające w tym przypadku, ponieważ głównym obiektem analizy jest zewnętrzny kształt dłoni.

d) Wybór największego konturu

Założenie, że największy kontur odpowiada dłoni - znaczne uproszczenie analizy obrazu w zamian na wrażliwość na błędy. Największy kontur przyjęto jako dłoń, ponieważ na przygotowanych obrazach była ona głównym obiektem po binaryzacji.

e) Wyznaczenie otoczki wypukłej - Convex Hull

Obliczenie wypukłego obwodu dłoni. Otoczka wypukła jest najmniejszym wypukłym kształtem obejmującym cały kontur dłoni. W kodzie obliczana jest za pomocą:

```
hull = cv2.convexHull(cnt)
```

f) Wyznaczanie defektów wypukłości - Convexity Defects

Wykrywanie defektów wypukłości - różnice pomiędzy rzeczywistym konturem dłoni a jej otoczką wypukłą. W praktyce zagłębienia te często odpowiadają przestrzeniom pomiędzy palcami.

Defekty wyznaczane są funkcją:

```
defects = cv2.convexityDefects(cnt, hull_indices)
```

Każdy defekt zawiera :

- s - indeks punktu początkowego,

- e – indeks punktu końcowego,
- f – indeks punktu najdalszego od otoczki,
- d – głębokość defektu.

Defekty wypukłości są kluczowym parametrem, ponieważ różne gesty mają różną liczbę i głębokość zagłębień pomiędzy palcami.

g) Operacja morfologiczna otwarcia

Usunięcie drobnego szumu z obrazu binarnego.

Otwarcie można zapisać jako:

$$A \circ B = (A \ominus B) \oplus B$$

gdzie A - obraz binarny, B - element strukturalny, $\ominus$ - erozja, $\oplus$ - dylatacja.

W projekcie zastosowano element strukturalny o rozmiarze 3×3 , który dobrano eksperymentalnie, ponieważ usuwał drobny szum bez zauważalnego zniekształcania kształtu dłoni.

h) Operacja morfologiczna zamknięcia

Wypełnienie małych dziur i scalenie fragmentów dłoni.

Zamknięcie można zapisać jako:

$$A \bullet B = (A \oplus B) \ominus B$$

gdzie A - obraz binarny, B - element strukturalny, $\ominus$ - erozja, $\oplus$ - dylatacja.

W projekcie zastosowano element strukturalny o rozmiarze 3×3 . Element 3×3 zastosowano, ponieważ dobrze wypełniał małe ubytki w masce, nie usuwając istotnych szczegółów konturu.

i) Ekstrakcja deskryptorów geometrycznych

W projekcie wykorzystano następujące deskryptory geometryczne:

- Udział pikseli obiektu wewnątrz konturu (`black_pixels_inside_contour_prc`) - określa stopień wypełnienia konturu przez obszar dłoni. Mnożnik 5 dobrano eksperymentalnie, aby zwiększyć wpływ tej cechy na odległość w klasyfikatorze k-NN.
- Składowa pozioma średniego kierunku defektów wypukłości (`mean_dir_x`) - średni kierunek rozmieszczenia zagłębień pomiędzy palcami względem ich środka.
- Składowa pionowa średniego kierunku defektów wypukłości (`mean_dir_y`) - analogiczna cecha opisująca pionowy rozkład defektów wypukłości.

- Średnia głębokość defektów wypukłości (`mean_depth`) – przeciętna głębokość zagłębień pomiędzy konturem dłoni a otoczką wypukłą. Wartość jest normalizowana względem pola konturu.
- Maksymalna głębokość defektów wypukłości (`max_depth`) – największe wykryte zagłębienie pomiędzy palcami. Cecha jest szczególnie przydatna przy rozróżnianiu gestów z rozstawionymi palcami.
- Odchylenie standardowe głębokości defektów (`std_depth`) – określa zróżnicowanie głębokości poszczególnych defektów wypukłości.
- Solidity – stosunek pola konturu dłoni do pola jej otoczki wypukłej:
`solidity = area_contour / area_hull`
Wartość bliska 1 oznacza zwarty kształt, z kolei mniejsze wartości bardziej luźny.
- Aspect Ratio – stosunek szerokości do wysokości prostokąta ograniczającego kontur:
`aspect_ratio = w / h`
- Extent – stosunek pola konturu do pola prostokąta ograniczającego:
`extent = area_contour / (w · h)`
- Circularity – współczynnik kolistości konturu:
`circularity = (4 · π · area_contour) / perimeter2`
Wartość równa 1 odpowiada idealnemu okręgowi. Mniejsze wartości oznaczają bardziej nieregularny lub wydłużony kształt dłoni.

Uzyskany zestaw cech tworzy wektor opisujący kształt gestu, który następnie wykorzystywany jest przez klasyfikator do porównywania podobieństwa pomiędzy próbkami.

j) Normalizacja

Głębokości defektów normalizowano względem pola konturu, aby ograniczyć wpływ rozmiaru dłoni na wynik klasyfikacji.

k) Obliczenie odległości euklidesowej

Porównanie podobieństwa wektora cech z bazą wzorców poprzez obliczenie różnicy pomiędzy próbkami z bazy a próbką treningową. W kodzie stosowana jest suma kwadratów różnic:

$$D(x, y) = \sum (x_i - y_i)^2$$

Jest to odległość euklidesowa bez pierwiastka. Pominięcie pierwiastka nie zmienia kolejności najbliższych próbek, dlatego wynik klasyfikacji pozostaje taki sam jak przy pełnej odległości euklidesowej:

l) Klasyfikacja gestu metodą k-NN

Rozpoznanie gestu na podstawie 4 najbardziej podobnych próbek treningowych. Wartość $k = 4$ dobrano eksperymentalnie, ponieważ dawała dobre wyniki i ograniczała wpływ pojedynczej błędnej próbki.

m) Walidacja modelu

Wielokrotny losowy podział danych treningowych/testowych (150 iteracji) do oceny skuteczności systemu. Na tej podstawie można obliczyć dokładność klasyfikacji:

```
accuracy = liczba_poprawnych_predykcji /  
liczba_wszystkich_probek
```

Dodatkowo wyniki mogą być przedstawione w postaci macierzy pomyłek. Wiersze macierzy odpowiadają klasom rzeczywistym, a kolumny klasom przewidzianym przez model. Macierz ta pozwala sprawdzić, które gesty są najczęściej klasyfikowane poprawnie, a które się ze sobą mylą.

3. Opis danych

W projekcie wykorzystano zbiór HGR1 pochodzący z bazy Hand Gesture Recognition. Baza zawiera obrazy dłoni przedstawiające gesty pochodzące z Polskiego Języka Migowego, Amerykańskiego Języka Migowego oraz znaki specjalne. Każda próbka w bazie składa się z obrazu RGB w formacie JPG oraz odpowiadającej mu maski binarnej skóry w formacie BMP (używanej tylko do wyznaczania zakresów HSV koloru skóry).

Tab 1. Charakterystyka wykorzystanej serii HGR1 - wybrane gesty

Cecha	Wartość
Liczba wszystkich wybranych obrazów	166
Liczba gestów	5
Tło w oryginalnej bazie	niekontrolowane
Oświetlenie	niekontrolowane
Obszar widoczności ze skórą	niekontrolowany

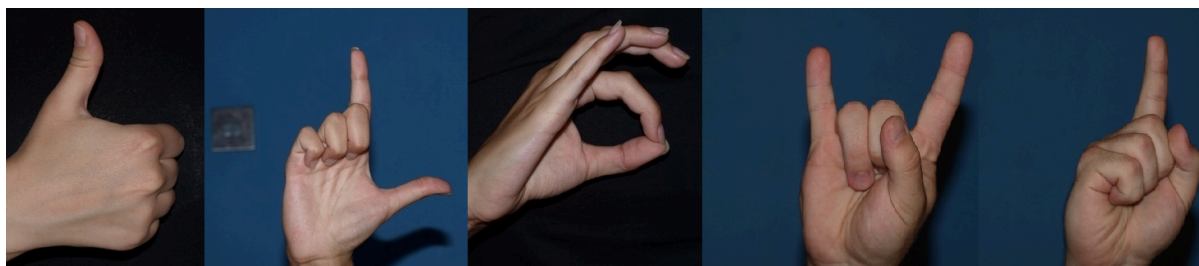
Na potrzeby projektu wykorzystano tylko gesty odpowiadające założeniom zadania. Analizowane klasy to: kciuk w górę, litera L, gest „okej”, gest „róg” oraz palec wskazujący skierowany w górę. Dane zostały dodatkowo przygotowane poprzez selekcję obrazów oraz kadrowanie dłoni do obszaru zainteresowania.

Zdjęcia zetykietowane według alfabetu migowego:

Kciuk w górę -	'1_P_hgr1_idx_y',
L -	'L_P_hgr1_idx_y'
Okej -	'O_P_hgr1_idx_y',
Róg -	'Y_P_hgr1_idx_y',
Palec wskazujący w górę -	'Z_P_hgr1_idx_y',

gdzie 'x' oznacza identyfikator, a 'y' numer zdjęcia danego identyfikatora.

Oprócz danych z bazy HGR1 przygotowano także własny zbiór zdjęć. Zdjęcia te przedstawiają te same klasy gestów i zostały wykonane z użyciem kamery smartfonowej. Zbiór własny służy do sprawdzenia, czy przygotowany system działa również na danych spoza bazy referencyjnej.



Rys 1. Przykładowe obrazy z bazy danych



Rys 2. Przykładowe zdjęcia z własnego zbioru

W celu uzyskania bardziej stabilnej oceny skuteczności zastosowano 150-krotne losowe powtarzanie podziału danych (Repeated Hold-Out Validation).

Losowane jest 90% zdjęć z dostępnej puli jako dane treningowe, 10% jako dane testowe 150 razy. Dane walidacyjne zastąpione są przez wielokrotny losowy podział.

Z obrazów treningowych wyciągane są cechy dłoni, które trafiają do 'feature_database'. Następnie k-NN porównuje nowe próbki do tych danych i dokonuje klasyfikacji.

4. Eksperymenty i analiza wyników

W ramach eksperymentów porównano działanie modelu YOLOv8_cls z autorskim systemem rozpoznawania gestów opartym na ekstrakcji cech geometrycznych oraz klasyfikacji metodą k-NN. Celem testów było sprawdzenie, jak na skuteczność klasyfikacji wpływają: jakość maski, przycięcie obrazu oraz usunięcie problematycznych próbek.

4.1 Oryginalne obrazy

W pierwszym eksperymencie sprawdzono skuteczność klasyfikacji na oryginalnych obrazach, bez selekcji próbek i bez przycinania dłoni do obszaru zainteresowania. Test przeprowadzono na różnych podziałach danych: 150

powtórzeń dla autorskiego systemu kNN oraz 5 powtórzeń dla modelu yolov8.

Tab 2. Parametry testu pierwszego

	kNN	yolov8
Ilość powtórzeń	150	5
Ilość zdjęć	166	166
Podział train/validate/test	90/10	70/20/10

Zbiór walidacyjny w przypadku modelu YOLOv8 służył do kontroli procesu uczenia oraz sprawdzenia, czy model nie dopasowuje się nadmiernie do danych treningowych.

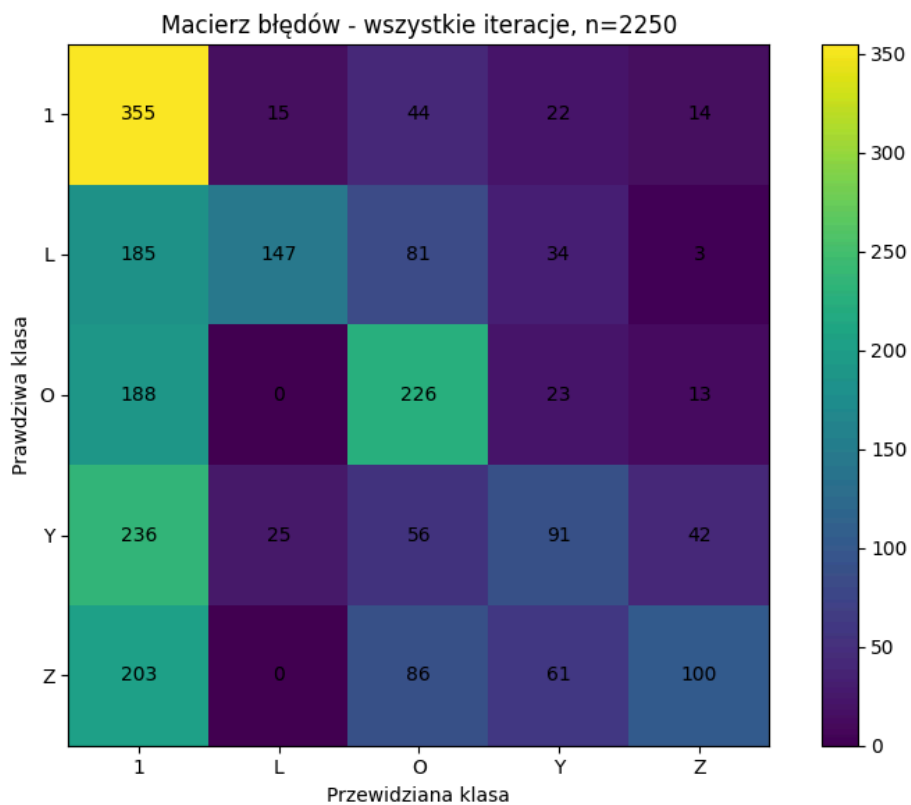
Tab 3. Wyniki testu dla kNN

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
919	1331	3.3228 s	0.1470	40.84

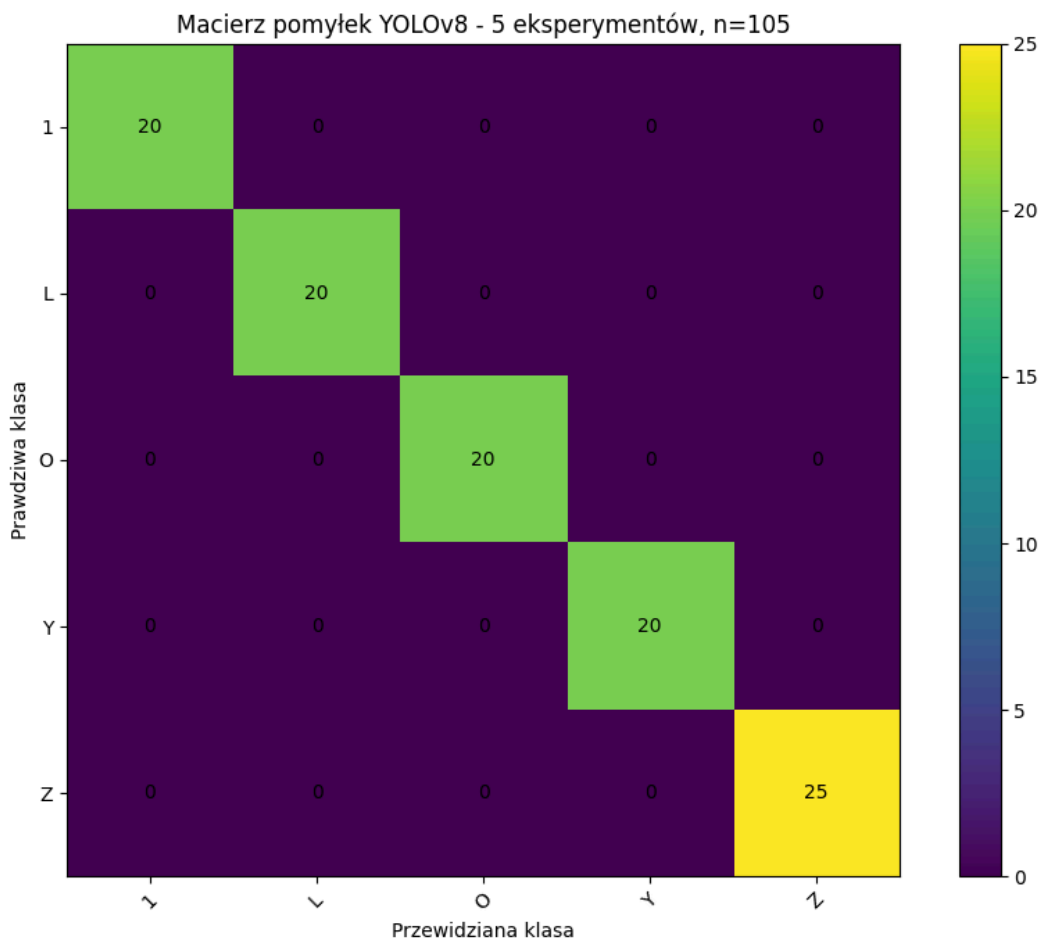
Tab 4. Wyniki testu dla YOLOv8

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
166	0	187.6847	0.7004 s	100

Jako, że na tych danych YOLOv8 osiągnął 100% dokładności, dalej nie będzie porównywany.



Rys 3. Macierz pomyłek dla systemu klasyfikacji kNN



Rys 4. Macierz pomyłek dla YOLOv8

Wyniki pokazują, że brak przycinania obrazów oraz obecność problematycznych próbek wyraźnie obniżają skuteczność systemu k-NN. Uzyskana dokładność jest jednak wyższa niż wynik losowy dla pięciu klas, co oznacza, że dobrany wektor cech częściowo poprawnie opisuje geometrię gestów. Model YOLOv8 uzyskał najwyższą skuteczność, jednak wymagał znacznie dłuższego czasu treningu i testowania.

4.2 Oryginalne maski

W drugim eksperymencie pominięto etap binaryzacji obrazu i wykorzystano gotowe maski z bazy danych. Celem było sprawdzenie, jak duży wpływ na wynik klasyfikacji ma jakość maski dłoni.

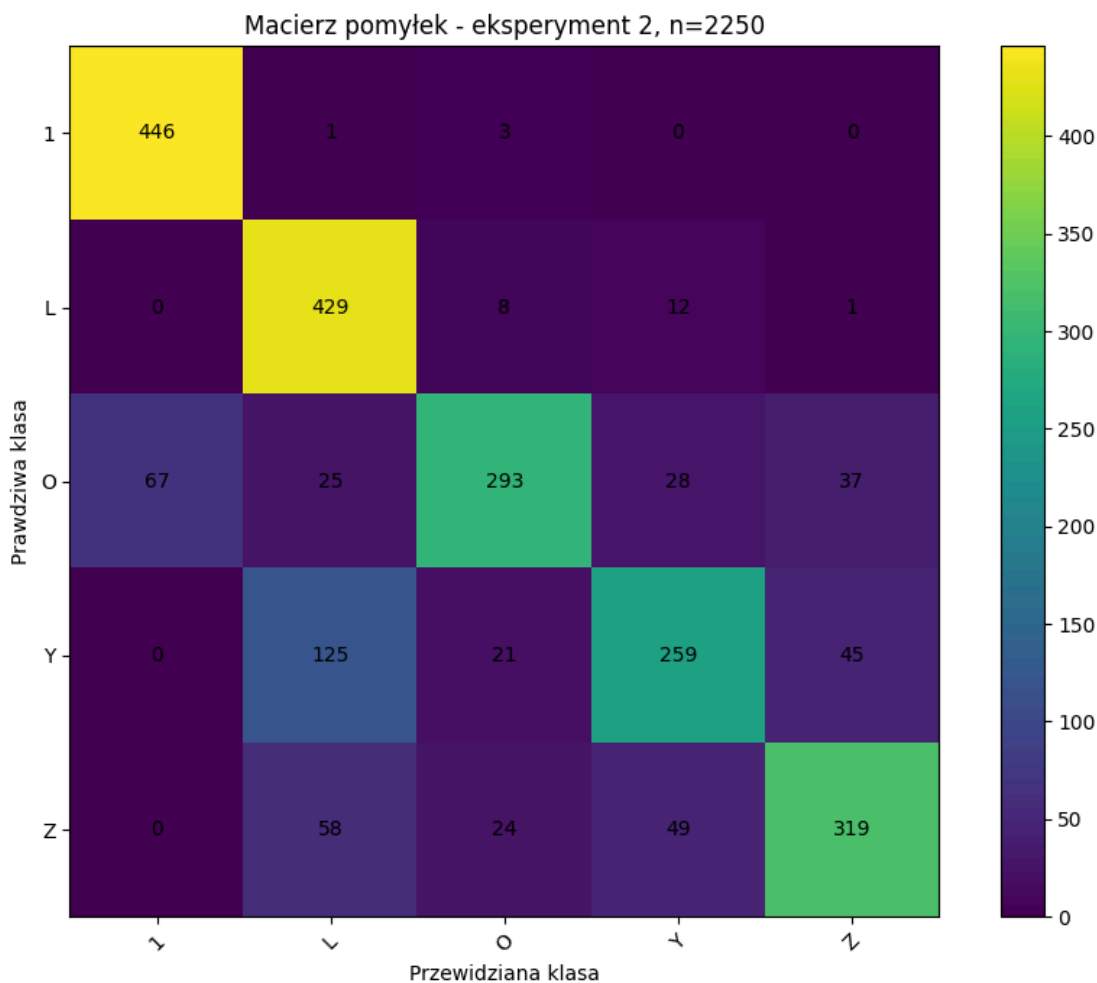
Tab 5. Parametry testu drugiego

	kNN
Ilość powtórzeń	150
Ilość zdjęć	166
Podział train/validate/test	90/10

Eksperyment pozwala bezpośrednio porównać wyniki systemu k-NN dla masek wygenerowanych automatycznie oraz masek przygotowanych w bazie.

Tab 6. Wyniki testu dla kNN z maskami z bazy

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
1746	504	0.7108 s	0.0879	77.60



Rys 5. Macierz pomyłek dla eksperymentu drugiego klasyfikacji kNN

Zastosowanie poprawnych masek znacząco poprawiło wynik klasyfikacji. Dokładność wzrosła z 40.84% do 77.60%, co pokazuje, że jakość segmentacji dłoni ma kluczowe znaczenie dla działania systemu opartego na cechach geometrycznych.

4.3 Przycięte obrazy

W trzecim eksperymencie sprawdzono wpływ przycięcia obrazu do założeń projektu, czyli pozostawienia widocznej głównie dłoni od nadgarstka. Pozwala to ograniczyć wpływ tła oraz dodatkowych fragmentów skóry na proces ekstrakcji cech.

Tab 7. Parametry testu trzeciego

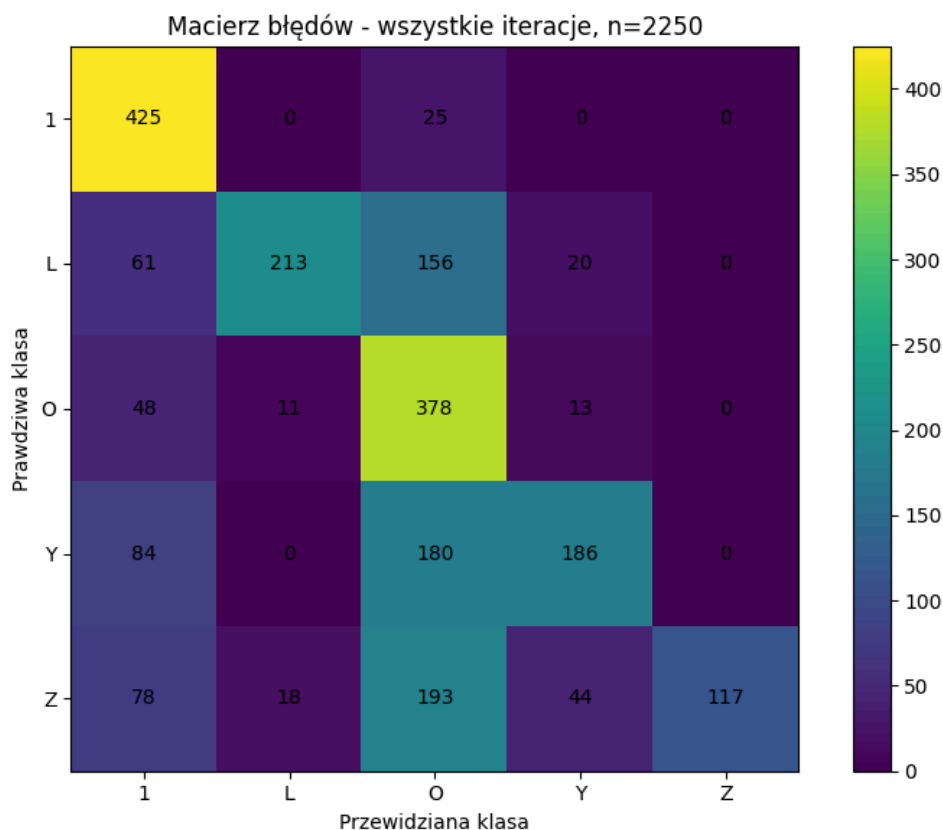
	kNN
Ilość powtórzeń	150
Ilość zdjęć	166

Podział train/validate/test

90/10

Tab 8. Wyniki testu dla kNN z przyciętymi zdjęciami

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
1319	931	1.4246	0.0695	58.62



Rys 6. Macierz pomyłek dla eksperymentu trzeciego klasyfikacji kNN

Przycięcie obrazów poprawiło skuteczność klasyfikacji względem pierwszego eksperymentu, jednak wynik nadal był niższy niż przy zastosowaniu gotowych masek z bazy. Oznacza to, że samo ograniczenie obszaru obrazu pomaga, ale nie rozwiązuje problemów wynikających z błędnej segmentacji dłoni.

4.4 Przycięte maski

W czwartym eksperymencie wykorzystano przycięte maski z bazy danych. Jest to wariant, w którym system pracuje na danych najlepiej

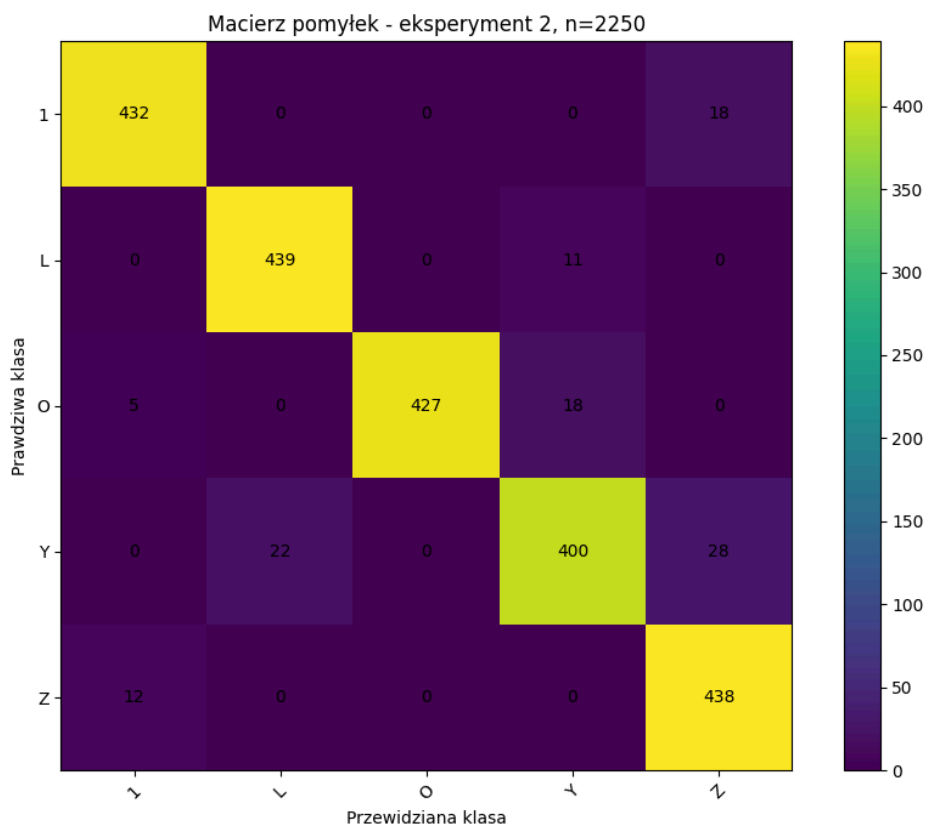
dopasowanych do założeń projektu: obszar obrazu jest ograniczony, a maska dłoni jest poprawna.

Tab 9 Parametry testu czwartego

	kNN
Ilość powtórzeń	150
Ilość zdjęć	166
Podział train/test	90/10

Tab 10. Wyniki testu dla kNN z przyciętymi maskami

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
2 136	114	0.5408	0.064	94.93



Rys 7. Macierz pomyłek dla eksperymentu czwartego klasyfikacji kNN

Połączenie przycięcia obrazu z użyciem poprawnych masek dało bardzo wysoki wynik klasyfikacji. Dokładność na poziomie 94.93% wskazuje, że przy dobrze przygotowanych danych klasyczny system oparty na cechach geometrycznych działa skutecznie.

4.5 Przycięte obrazy + usunięcie problematycznych zdjęć

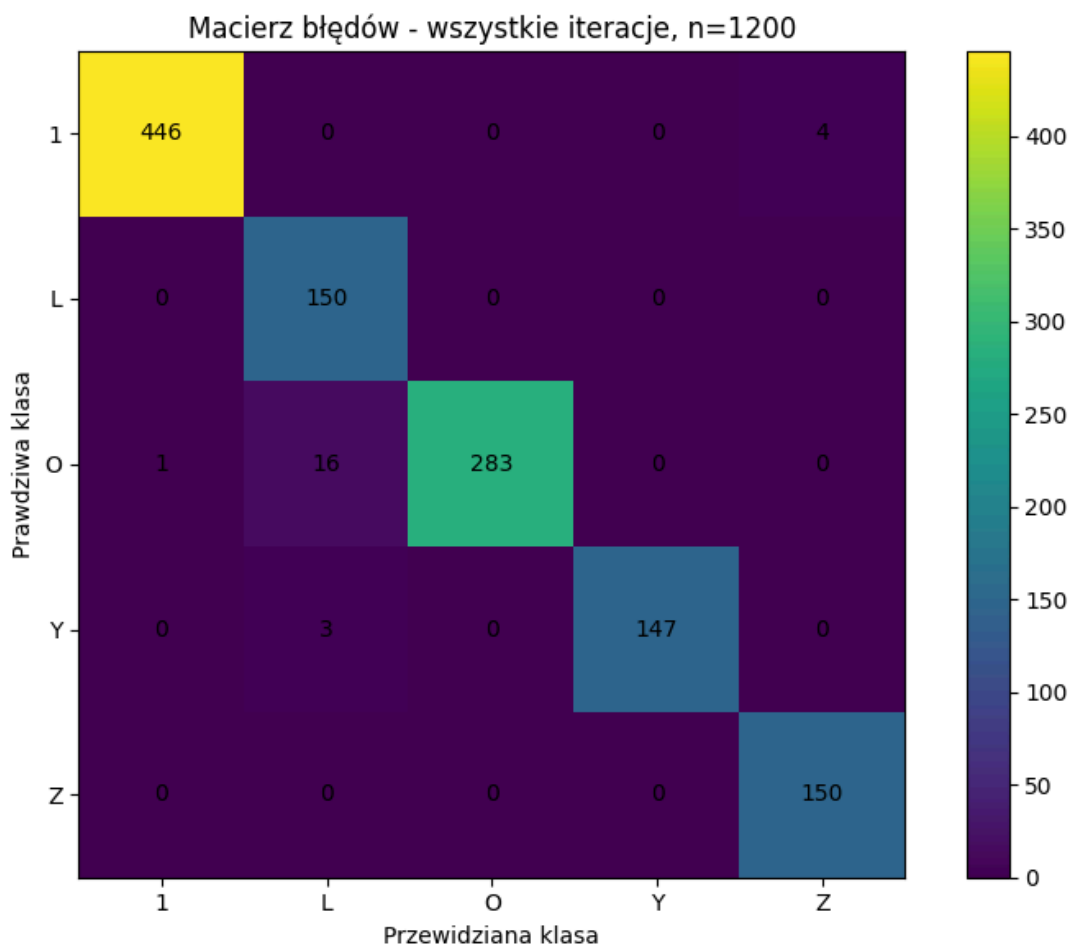
W piątym eksperymencie usunięto obrazy, dla których system miał problem z poprawnym utworzeniem maski. Następnie wykorzystano przycięte obrazy oraz ręcznie przygotowane maski z oryginalnych zdjęć. Celem było sprawdzenie, jak selekcja danych wpływa na końcowy wynik klasyfikacji.

Tab 11. Parametry testu piątego

	kNN
Ilość powtórzeń	150
Ilość zdjęć	105
Podział train/test	90/10

Tab 12. Wyniki testu dla kNN z przyciętymi maskami i usuniętymi zdjęciami

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
1 176	24	1.1507	0.0416	98.00



Rys 8. Macierz pomyłek dla eksperymentu piątego klasyfikacji kNN

Usunięcie problematycznych próbek dodatkowo poprawiło skuteczność klasyfikacji. Wynik 98.00% pokazuje, że największy wpływ na działanie systemu mają jakość danych wejściowych oraz poprawność przygotowania masek.

4.6 Trening na bazie, test na danych z zewnątrz - bez usuwania problematycznych zdjęć

W kolejnym eksperymencie model został nauczony na danych pochodzących z bazy referencyjnej, a następnie przetestowany na zewnętrznych zdjęciach wykonanych poza bazą HGR1. W tym wariancie klasyfikacja była wykonywana na podstawie masek wygenerowanych automatycznie z oryginalnych zdjęć. Nie usuwano przy tym zdjęć problematycznych, dla których segmentacja dłoni mogła przebiegać niepoprawnie.

Można było założyć, że wyniki tego testu będą słabsze, ponieważ zdjęcia zewnętrzne różnią się od danych treningowych warunkami oświetlenia, tłem oraz wyglądem dłoni. W takich przypadkach zakres HSV

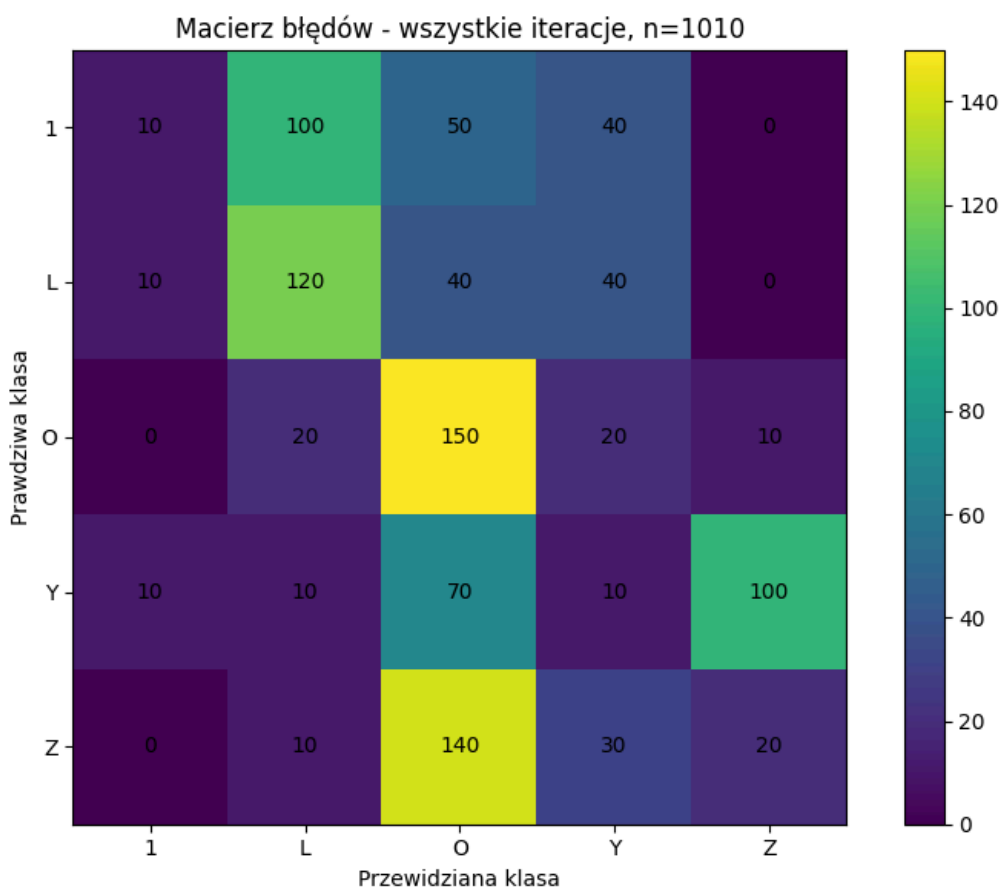
wyznaczony na podstawie bazy treningowej może nie przenosić się dobrze na nowe dane.

Tab 13. Parametry testu szóstego

	kNN
Ilość powtórzeń	10
Ilość zdjęć	101

Tab 14. Wyniki testu nr 6

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
31	70	0.3910	0.0623	30.69



Rys 9. Macierz pomyłek dla eksperymentu szóstego klasyfikacji kNN

Uzyskane wyniki potwierdzają wcześniejsze założenie. Dokładność klasyfikacji była stosunkowo niska, choć nadal wyższa niż wynik losowy. Po bliższej analizie wygenerowanych masek można zauważyć, że w wielu przypadkach segmentacja dłoni była wykonana nieprawidłowo. System błędnie zaznaczał fragmenty tła albo pomijał istotne części dłoni, co

bezpośrednio wpływało na jakość wyznaczanych cech i końcową decyzję klasyfikatora.



Rys 10. Przykładowe stworzone maski

4.7 Trening na bazie, test na danych z zewnątrz - jako maski

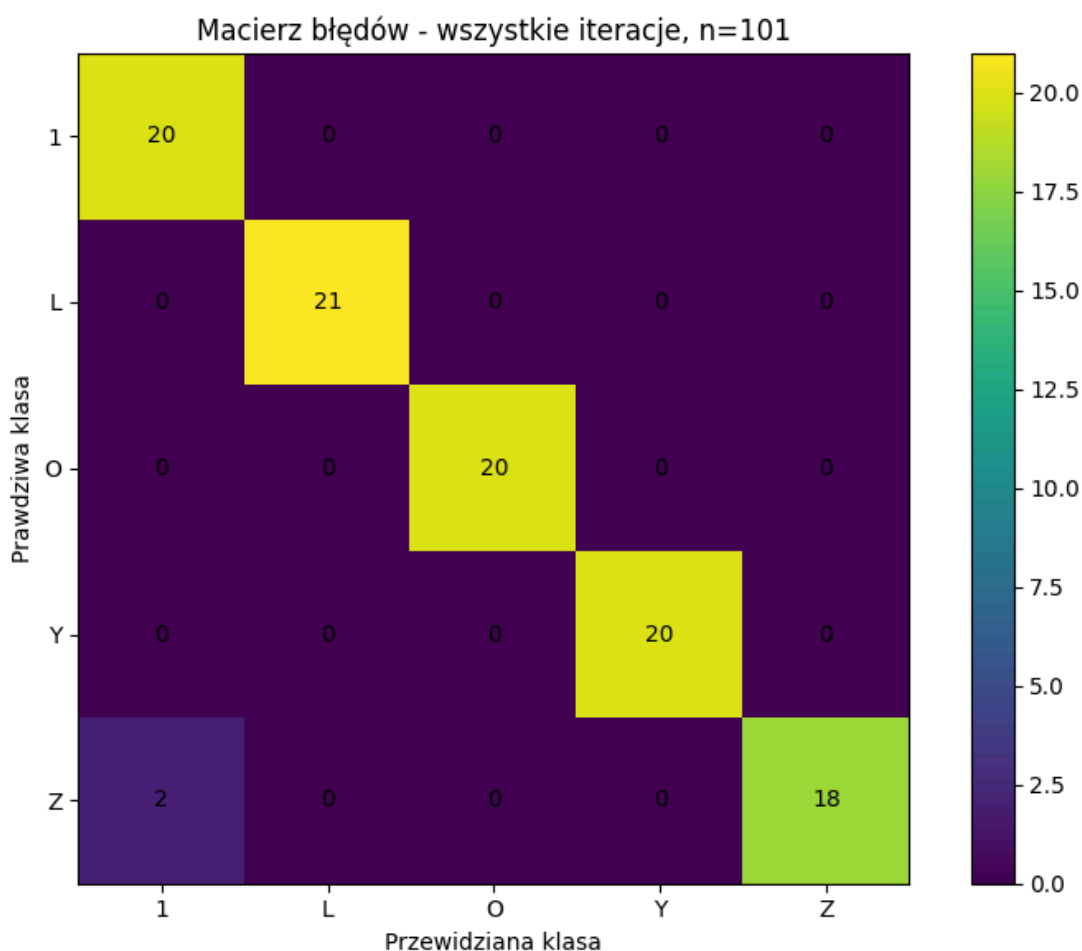
W ostatnim eksperymencie przygotowany system jest sprawdzony na danych spoza bazy referencyjnej. Dane treningowe będą pochodziły z wyselekcjonowanego i przyciętego zbioru HGR1, natomiast jako dane testowe wykorzystane zostaną własne zdjęcia przedstawiające te same klasy gestów. Tym razem maski są uprzednio przygotowane i poprawnie zbinaryzowane. Eksperyment pozwoli ocenić, czy system zachowuje skuteczność dla dłoni z innej bazy danych, kiedy maska jest poprawna.

Tab 15. Parametry testu szóstego

	kNN
Ilość powtórzeń	1
Ilość zdjęć	101

Tab 16. Wyniki testu dla kNN z przygotowanymi maskami

L. poprawnych	L. niepoprawnych	Średni czas treningu [s]	Średni czas testu [s]	Dokładność [%]
99	2	1.1627	0.0619	98.00



Rys 11. Macierz pomyłek dla eksperymentu siódmego klasyfikacji kNN

Eksperyment potwierdza wcześniejszą tezę: system zachowuje się poprawnie, kiedy maski są prawidłowe.

5. Wnioski

Eksperymenty wykazały, że największym wyzwaniem dla stworzonego systemu k-NN była automatyczna segmentacja dłoni. Gdy testowano go na oryginalnych zdjęciach, efektywność okazała się niska. System często niewłaściwie generował maski – czasem zaznaczał fragmenty tła, pomijał części dłoni lub nie radził sobie dobrze przy zmiennym oświetleniu. To szczególnie uwidaczniało się podczas testów na zdjęciach wykonanych na zewnątrz, gdzie dokładność spadła aż do 30,69%.

Klasyfikacja znacznie poprawiała się, jeśli korzystano z dopracowanych masek. W przypadku wyciętych masek i usunięcia problematycznych obrazów udało się osiągnąć wysoką skuteczność, sięgającą 98%. Pokazuje to, że dobrane cechy geometryczne oraz klasyfikator k-NN dobrze rozpoznają gesty, ale tylko wtedy, gdy dłoń jest poprawnie wyodrębniona z obrazu.

W porównaniu z modelem YOLOv8 autorski system k-NN był prostszy, szybszy i łatwiejszy do interpretacji. YOLOv8 osiągnął jednak lepszą skuteczność na oryginalnych obrazach i był bardziej odporny na tło oraz jakość danych wejściowych. Wadą YOLOv8 był znacznie dłuższy czas treningu oraz większe wymagania obliczeniowe.

System k-NN sprawdza się dobrze dla danych przygotowanych i poprawnie zbinaryzowanych, ale słabo radzi sobie z nowymi zdjęciami, jeżeli maska jest tworzona automatycznie. Najważniejszą wadą systemu jest niedokładna segmentacja dłoni. Można by to polepszyć, poprzez zwiększenie liczby zdjęć w bazie oraz dopracowaniu systemu, aby bardziej dynamicznie dobierał zakresy HSV.

6. Bibliografia

- <https://gist.github.com/Dhruv454000/dce6491280e09ff8d920ed46fc625889>
- https://docs.opencv.org/4.x/d7/d1d/tutorial_hull.html
- https://docs.opencv.org/3.4/d5/d45/tutorial_py_contours_more_functions.html
- https://www.researchgate.net/publication/392161160_Hand_Tracking_and_Gesture_Recognition_for_Human-Computer_Interaction
- <https://sun.aei.polsl.pl/~mkawulok/gestures/>